

Last modified on

Sports Data Soccer API - Match Fitness Feed (MA24)

Get cumulative tracking data on teams and players during a given match, with a breakdown for player fitness by movement type.

Overview

The Soccer **Match Fitness feed** provides a way for you to gain additional context to the match around player fitness. The feed splits movement into a number of statistics and attributes, largely based on movement speed, distance, runs and sprints. Statistics are delivered under the **player** and **teamStats** nodes - you can view a list of available statistics under **Statistic Types**.

Note: This feed will launch with limited availability. Please ask your Account Manager for information about competition availability and about commercial legalities around this feed.

Definitions: Movement, Distance, Sprints

Use these definitions to give context to and understand the data points and statistics returned by the feed.

SportVU Covered Competitions

SPEED	MOVEMENT DEFINITION
0-1 kmh	standing
1-6 kmh	Walking
6-15 kmh	Jogging
15-20 kmh	Running
20-25 kmh	High-speed Running
>25 kmh	Sprinting

Other Tracking Provider Competitions

SPEED	MOVEMENT DEFINITION
<7 kmh	Walking
7-14 kmh	Jogging
14-17 kmh	Low-speed Running
17-21 kmh	High-speed Running
21-24 kmh	Low-speed Sprinting
>24 kmh	High-speed Sprinting

Distance Covered

The distance covered by a player is added when they have moved at least one metre from their previously-detected position. This means that the data given for distance covered accounts for different scenarios.

Example scenarios:

- A player takes 1 step shorter than 1 metre in one direction and 1 step back in the opposite direction (where the step is equally as short) = the given distance will not increase
- A player moves 80 centimetres in one direction and then another 30 centimetres in the same direction = 1.10 metres will be added to the given distance

Sprints: Number Of Sprints And Additional Sprint Sequences

- Sprint sequence start: A sprint event is registered each time that the player's movement speed goes into one of the sprint intervals defined above ('Low-speed Sprinting').
- Sprint sequence end: The sequence of sprint ends when the player's movement speed goes below the lower sprint interval threshold.
- Additional sprint sequences: If a second sprint sequence starts within 25 frames after the end of the previous sprint, only one sprint will be counted. Attacking and defensive sprints are separated based on direction.

Required Compliance:

To comply with our API security and best practice logic for your API calls please refer to our [FAQs](#). We also recommend that you take a look at our [API Overview and Authentication Guide](#). You will learn the basics of how our API works and key concepts to get started.

How Often Should I Poll The Feed?

We have provided some guidelines to help ensure that you call the feed only as often as you need to. Make sure that you don't exceed the rate confirmed with us - this can vary depending on whether you make requests as a B2B or B2C outlet - if you need more information on this, contact your Stats Perform representative.

UPDATED	POLLING FREQUENCY							
	TIER 15	TIER 12+14	TIER 4	TIER 11+13	TIER 8+9+10	TIER 6+7	TIER 5	TIER 1+2+3
On valid events during the match	Once every 60 seconds							

Match Fitness (MA24) Guide

USAGE	FEED
GET match fitness data by fixture UUID (path parameter method)	/soccerdata/matchfitness/{outletAuthKey}/{fixtureUu

If your request URL is as follows - that is, it does not contain a fixture UUID - the feed returns an error:
/soccerdata/matchfitness/{outletAuthKey}?_fmt={xml|json}&_rt={b|c}

Query Parameters And Example Request Parameters

This feed also makes use of the **Global query parameters**, and you can combine them with the following query parameters (some of which can be used in conjunction with each other). However, you must query the **/matchfitness** feed by fixture UUID.

The ability to query by a legacy Opta ID is not provided in the Match Fitness feed.

How to use query parameters and make successful requests:
In our guides, we use HTTPS syntax for our example GET requests (rather than cURL, for example). All URLs are case-sensitive - this means that URL strings must contain the correct case and operators. You MUST format your requests correctly and include certain information; otherwise, an error will be returned. You can find information about the API, including the base URL ({protocol}://{requestDomain}/{feedResource}), in the **main guide** and **API Overview and Authorisation guide**.

- Include the following information in the URL or header and make sure it is correct for your outlet: your unique and valid {outletAuthKey} this must be after the Outlet Authentication Key (and endpoint, if applicable). for _rt={mode} (operating mode) and _fmt={dataFormat} (response format), entity query parameter and UUID.
- Begin the query parameter string with ? (question mark) - this must be after the Outlet Authentication Key (and endpoint, if applicable). To build up a query string, make sure that each query parameter is separated by the & (ampersand) operator. Values must be separated by the correct operator, for example & (multiple values, 'AND'), or - if a parameter supports it - , (multiple values, 'OR') or ! ('NOT').
- Use the required _ (underscore) for any parameters listed with this prefix.
- Remove any placeholder value curly braces {} from your calls (we include these as an example only).
- If using the JSONP format (_fmt=jsonp) you MUST use it in combination with the callback parameter (_clbk) and a fixed value (or where you use a different value in a different instance, you must do this as little as possible). Be aware that common JavaScript frameworks, such as jQuery, can default to use randomly-generated function names - you must make sure that these are overridden and define a fixed function name instead.

Query parameter	Value and description
{fixtureUuid} Fixture UUID	GET match fitness data by by fixture UUID (path parameter method). Pass the fixture UUID in the request URL directly as the {assetId} (asset/resource ID). GET https://api.performfeeds.com/soccerdata/matchfitness/{outletAuthKey}/{fixtureUuid?_rt={mode}&_fmt={dataFormat}

Sample Calls

These sample calls let you see an example URL and the response - click on the links below (hyperlinks open in a new window).

- **[XML](#)**

- **JSON**

Example Call And Response

In this example, the API call is a request to return match fitness data for fixture UUID `7ijuqohwgmplbxjdj1625sxwfe` from the `/matchfitness` endpoint.

```
GET https://api.performfeeds.com/soccerdata/matchfitness/{outletAuthKey}/7ijuqohwgmplbxjdj1625sxwfe?&_rt=b
&_fmt=xml
```

Feed Output

Here, you can find an explanation of all possible response fields and data. This is XML but fields in the feed response are similar for JSON.

▶<matchFitness> (XML only)

▶<matchInfo>

▶<description>

▶<sport>

▶<ruleset>

▶<competition>

▶<country>

▶<tournamentCalendar>

▶<stage>

▶<series>

▶<contestants> (XML only)

▶<contestant>

▶<country>

▶<venue>

▶<liveData>

▶<matchDetails>

▶<periods> (XML only)

▶<period>

▶<suspensions>

▶<suspension>

▶<scores>

▶<ht>

▶<ft>

▶<et>

▶<pen>

▶<total>

▶<lineUp>

▶<player>

▶<stat>

▶<teamOfficial>

▶<teamStats>

▶<stat>

Statistic Types

Statistics are split between and delivered under the following elements (where available):

- **player** (in the team/contestant)
- **teamStats** (contestant)

Within these elements, each statistic is shown as a **stat** and its **type** and **value** will appear under it, if it is available.

Click on the buttons below to see the complete lists of player and contestant (team) and statistics. For an explanation of the different types of movement speed, distances covered and sprint sequences, see: [Definitions: Movement, Distance, Sprints](#)

Player Stats

▶**Player stats - names and descriptions**

Team Stats

For these stats, each value is a total that is calculated based on individual player stats. For example, the value for 'distance' is the sum of the 'distance' values given for each individual player.

▶**Team stats - names and descriptions**